Attention Parity Beats ReZero: Init-Preserving Depth-Wise Routing for Recurrent Memory in Pretrained LLMs

Anonymous Author(s)
Affiliation
Address
email

Abstract

Augmenting a pretrained Transformer with off-sequence recurrent memory requires choosing how the per-position memory readout $m^t \in \mathbb{R}^{S\times d}$ enters the residual stream at every depth. Three routing primitives are concretely available: a per-sublayer *ReZero* scalar gate (SIMPLE_GATE), a Block-Attention-Residual delta-source router *without* parity-preserving init (ATTENTION_BASE), and the same router *with* parity-preserving init (ATTENTION_PARITY). We ask: under matched compute, seed, and training data, which primitive learns chain-specific recurrent memory most sample-efficiently in a frozen pretrained LLM? We train all three on a frozen Qwen3-0.6B backbone recurrently on ≈113M tokens of PG-19 books and TV dialogue using truncated back-propagation through time, and evaluate history-specificity via a chain-shuffle gap Δ_{sh}. Soft ATTENTION_PARITY reaches SIMPLE_GATE's asymptotic shuffle-gap plateau in approximately $2/5$ the steps and continues climbing; ATTENTION_BASE spends most of its compute budget reabsorbing a 34-nat initialisation perturbation and never surpasses the bare backbone at matched compute. Two held-out mechanistic probes confirm the difference is structural: parity-init opens $3\times$ more routing mass on the memory source and produces a held-out next-token loss that is $2.5\times$ more causally sensitive to memory perturbations than the no-parity baseline. The routing-mass gap localises to mid-network sublayers. We conclude that parity-preserving initialisation is a first-class *architectural* decision for off-sequence memory routing—not a numerical safety device—and that without it, the same router architecture fails to open a memory channel under realistic compute budgets.

1 Introduction

A pretrained Transformer's residual stream is a finely-balanced construction. Augmenting it with an off-sequence memory readout $m^t \in \mathbb{R}^{S\times d}$, produced by an external recurrent state $M_{\text{c}} \in \mathbb{R}^{K\times d}$ that crosses session boundaries [4, 11, 3, 7, 12, 13], requires choosing *how m^t enters the residual stream at every depth*. Three routing primitives are concretely available to the architecture-design literature:

- SIMPLE_GATE — a per-sublayer ReZero scalar gate [2], $h \leftarrow h + g_\ell\, m^t$ with g_ℓ initialised at zero. Bit-exact step-0 backbone parity at all sublayers.
- ATTENTION_BASE — the Block-Attention-Residual router [5] with m^t registered as a foundational source b_{-1} alongside per-block delta sources $b_0, \ldots, b_{n-1}$. A uniform-softmax init perturbs final-layer logits by $\sim$34 nats.
- ATTENTION_PARITY — the same router with two coupled changes: (i) the value pool stores cumulative hidden-state checkpoints (so the residual stream is recoverable as a one-hot

Submitted to 40th Conference on Neural Information Processing Systems (NeurIPS 2026). Do not distribute.

select on the most-recent slot), and (ii) two scalar bias parameters initialise the softmax one-hot on the most-recent source. Bit-exact step-0 parity at strict ± 32 bias; the *soft* variant uses ± 4 bias, sacrificing exact parity for non-saturated softmax gradients from step 1.

Empirical question. Under matched compute ($\approx$113M Qwen tokens of PG-19 + TV, $6{,}000$ TBPTT steps, single H100, identical seed), which of these three primitives learns chain-specific recurrent memory most sample-efficiently on a frozen Qwen3-0.6B backbone?

Answer. Soft ATTENTION_PARITY reaches SIMPLE_GATE's asymptotic shuffle-gap plateau in roughly $2/5$ of the steps and continues climbing. ATTENTION_BASE spends most of its compute budget reabsorbing a 34-nat init perturbation and at matched compute does not surpass the bare backbone on either the aggregate memory-help or the history-specificity diagnostic. Two held-out mechanistic probes confirm that the difference is structural, not just numerical: parity-init opens $3\times$ more routing mass on the memory source and produces held-out loss that is $2.5\times$ more causally sensitive to memory perturbations than the no-parity baseline. The routing-mass gap concentrates in mid-network sublayers.

Contributions.

1. A clean architectural decomposition of the three routing primitives that fits standard published primitives (ReZero scalar gate, Block-AttnRes delta-source router, parity-preserving cumulative pool) and reduces the choice between them to two scalar init-bias parameters and one pool-construction switch (§3).
2. A bit-exact init-parity verification table: SIMPLE_GATE and strict ATTENTION_PARITY both achieve $|\Delta_{\text{logit}}| < 10^{-6}$ against the bare backbone, while ATTENTION_BASE perturbs by 34 nats (Table 1).
3. A matched-compute, matched-seed head-to-head on Qwen3-0.6B trained recurrently on PG-19 + TV dialogue: soft ATTENTION_PARITY leads SIMPLE_GATE by $1.6\times$–$3.8\times$ on the history-specificity gap Δ_{sh} at every in-trainer eval point and asymptotically surpasses SIMPLE_GATE's plateau (§5.1).
4. Two held-out mechanistic probes that target router behaviour rather than output: a *routing-mass trace* on 156 score positions across 40 held-out chains, and a *counterfactual sensitivity* probe that perturbs slots in the prior memory state and measures the held-out loss response. Parity-init opens $3\times$ more mass and produces $2.5\times$ more causal sensitivity than no-parity; without parity init no such structure forms (§5.3).
5. A clean negative result for ATTENTION_BASE: under matched compute, the same Block-AttnRes router *without* parity-preserving init does not learn a memory channel. The architectural primitive needs the init.

2 Background and related work

Recurrent memory in Transformers. Transformer-XL [4] caches past hidden states and attends over them within a fixed window; this preserves up to $O(N)$ context but does not give a compact compressed state across session boundaries. Compressive Transformer [11] adds a coarse memory bank with hand-designed compression operators; the resulting state is differentiable but the compression schedule is fixed. Recurrent Memory Transformer [3] prepends a small bank of memory tokens to the input sequence and trains them recurrently across segments; its narrow channel and prepend-to-sequence design recreate the lost-in-the-middle pathology at long horizons and require multi-stage warm-ups. Block-Recurrent Transformer [7] introduces a learned gate to merge the recurrent state with the per-block hidden, but the gate is brittle to optimise and exhibits the channel-collapse failure mode when trained on natural data. Memorising Transformers [12] retrieve from a key-value external memory but do not propagate gradient into the memory itself. LRMT [13] learns a hierarchical compressed memory with explicit gating; it shows promising long-horizon results but exhibits horizon-overfit when training-time TBPTT is much shorter than eval horizons.

Block attention residuals. Recent work [5] proposes a generalisation of the residual stream in which every transformer sublayer receives, as input, a softmax-weighted mix over a pool of *block-level sources*—the input embedding, prior block outputs, and a running intra-block partial. This

depth-wise routing pool is what we exploit to inject the memory readout off-sequence: registering the per-position memory readout $m^t \in \mathbb{R}^{S\times d}$ as a foundational source b_{-1} alongside the embedding source b_0 yields a routing primitive that adds an *architecturally-stable*, but learnable, depth-wise pathway from memory to every sublayer.

Building blocks the architecture borrows. The two-stage extraction stack is a Perceiver [8]-style fixed-bandwidth latent, where a learnable query bank cross-attends over the longer raw sequence and is then iteratively refined. The SIMPLE_GATE routing primitive uses a ReZero-style scalar gate [2]: a single scalar per sublayer, initialised at zero, multiplying the additive memory readout. The state-space-model literature, e.g. Mamba [6], also maintains a token-level recurrent state but commits to it *before* seeing the next token; we instead update the chain state at session boundaries only, which lets the judge step condition on the entire just-completed session before deciding what to remember.

Init-parity tricks elsewhere. The parity-preserving init we exploit here is conceptually adjacent to ReZero's zero-gate scaling [2], to LayerScale's zero-init residual gate, and to mixture-of-experts soft routers initialised away from saturation. The novelty in our setting is that the source the router must *not* put mass on at init (the memory source b_{-1}) is the same source that the router must *eventually* put non-trivial mass on for the architecture to be useful. Strong negative bias on a single softmax entry produces a vanishing gradient on its pseudo-query at saturated init, which is why the soft variant (± 4) substantially outperforms the strict variant (± 32) on a realistic compute budget.

3 The Memory Residuals architecture

The architecture is described in detail in the position paper [1]. We summarise the equations here, taking care to flag the design choices that turn out to be empirically load-bearing in §5.

Memory state. Each chain (a book, a show, or a conversation) carries a single fixed-size matrix $M_{\text{c}} \in \mathbb{R}^{K\times d}$, where $K = 128$ slots and d matches the backbone hidden size (1024 for Qwen3-0.6B). The state is updated once per session boundary, never per token.

Stage 1 (extract). A learnable bank of queries $M_{\text{in}} \in \mathbb{R}^{K\times d}$ cross-attends over the just-completed session $C_t \in \mathbb{R}^{N_t\times d}$ to distill it into a K-slot candidate

$$M_{\text{new}} \;=\; \text{Softmax}\!\left(\frac{M_{\text{in}} W_Q^{\text{ext}} \big(C_t W_K^{\text{ext}}\big)^\top}{\sqrt{d}}\right) C_t W_V^{\text{ext}}. \tag{1}$$

With extraction depth $L_E > 0$, a Perceiver-style stack lets M_{new} re-query C_t for L_E extra rounds; we use $L_E = 4$ in all reported runs.

Stage 2 (judge). A *separate* cross-attention with learnable judging queries $M_{\text{judge}} \in \mathbb{R}^{K\times d}$ attends over the concatenated pool $P = [M_{\text{c}}^{t-1} \parallel M_{\text{new}}] \in \mathbb{R}^{2K\times d}$:

$$M_{\text{c}}^{t} \;=\; \text{Softmax}\!\left(\frac{M_{\text{judge}} W_Q^{\text{j}} \big(P W_K^{\text{j}}\big)^\top}{\sqrt{d}}\right) P W_V^{\text{j}}. \tag{2}$$

Because the softmax is computed across the $2K$ row dimension, old and new content compete for each output slot's attention mass: a mundane session lets the old memory pass through, a salient session overwrites. This *zero-sum forgetting defence* is the single reason the architecture does not need RMT-style warm-up tricks. A small RMSNorm applied after the judge cross-attention prevents $\|M_{\text{c}}\|_F$ from drifting after ~ 10 recurrent unrolls.

Off-sequence read. At inference into the next session, every position x_i cross-attends M_{c} once to produce a per-position readout

$$m^t \;=\; \text{Softmax}\!\left(\frac{X W_Q^{\text{r}} \big(M_{\text{c}} W_K^{\text{r}}\big)^\top}{\sqrt{d}}\right) M_{\text{c}} W_V^{\text{r}} \in \mathbb{R}^{S\times d}. \tag{3}$$

Because m^t has the same shape as any sublayer output, it slots into the residual stream cleanly via three different routing primitives.

Table 1: Init parity verification on Qwen3-0.6B (bf16). Maximum per-token absolute logit deviation between each augmented model at step 0 and the bare backbone, on a 30-prompt batch with 27 tokens of prefix. Bit-exact parity ($\Delta < 10^{-6}$) is the requirement for "additive on top of the pretrained behaviour" to hold. See `paper_artifacts/eval/init_parity_test.json` for the machine-readable record.

mode	no memory	with memory
SIMPLE_GATE (ReZero, g_ℓ=0)	0.000	0.000
ATTENTION_BASE (uniform delta pool)	34.5	34.4
ATTENTION_PARITY (cumulative pool + $\pm$32 bias)	0.000	0.000
ATTENTION_PARITY (soft, $\pm$4 bias)	$\approx 10^{-5}$	$\approx 10^{-5}$

Routing primitive A: SIMPLE_GATE. A per-sublayer ReZero-style scalar gate g_ℓ injects m^t into each sublayer input, $h_\ell^{\text{pre}} = h_{\ell-1}^{\text{post}} + g_\ell m^t$, with g_ℓ initialised at zero. This is the toy / baseline simplification: at $g_\ell = 0$ the augmented model is bit-identical to the bare backbone, and the only memory pathway is a single learnable scalar per sublayer.

Routing primitive B: ATTENTION_BASE. A full Block AttnRes router [5] treats the memory readout m^t as an additional *delta source* b_{-1} alongside the per-block delta sources $b_0, \ldots, b_{n-1}$. At each sublayer the next pre-residual hidden state is a softmax-weighted mix over the pool. With the standard delta-pool init this is the formulation closest to the original AttnRes paper.

Routing primitive C: ATTENTION_PARITY. The same router as B, but with two coupled changes that make the init parity-preserving: (i) the value pool stores cumulative hidden-state checkpoints $b_0 = h_0$, $b_k = h_k$, plus the running intra-block partial $h_{n,i-1}$ (so the residual stream $h = b_0 + \sum_k b_k + h_{n,i-1}$ can be expressed as a one-hot selection on the most-recent slot), and (ii) the router carries two scalar bias parameters `mem_bias` and `recent_bias` that initialise the softmax one-hot on the most-recent source. We additionally study a **soft** variant of ATTENTION_PARITY in which the bias magnitudes are reduced from the strict-parity $\pm$32 to $\pm$4, sacrificing exact bit-identical step-0 logits in exchange for non-saturated softmax gradients on the per-source pseudo-queries from step 1.

Init parity. Table 1 reports the maximum per-token logit deviation between each augmented model and the bare Qwen3-0.6B backbone at step 0, in bf16, on a 30-prompt 27-history calibration batch. SIMPLE_GATE and the strict-parity variant of ATTENTION_PARITY achieve bit-exact agreement; the soft variant ($\pm$4) drifts by $\sim 10^{-5}$ logit; ATTENTION_BASE's uniform-softmax delta pool perturbs by up to 34 nats.

The recurrent training loop. Truncated-BPTT, k sessions per window, plain next-token NLL summed over the k sessions in the window so every M_c^t receives at least one downstream gradient; memory dropout and context dropout are applied only at training time and only outside the recurrent state update, so the dropout schedule cannot poison M_c. Pseudocode in Appendix A.

4 Experimental setup

Backbone. All experiments use the publicly-released Qwen3-0.6B checkpoint [10] as the frozen pretrained backbone. Memory parameters are added on top (§3); the optimiser groups are split, with the memres parameters trained at $\eta_m = 3\times10^{-4}$ and the backbone at $\eta_b = 3\times10^{-5}$, AdamW $(\beta_1, \beta_2) = (0.9, 0.95)$, weight decay 0.1, cosine schedule with 200 warmup steps and 5% minimum LR ratio.

Memory hyperparameters. $K = 128$ slots, $L_E = 4$ extraction-depth refinement, judge RMSNorm enabled, all ATTENTION_* routers using $N = 8$ blocks of Qwen3-0.6B.

Data. **Stage 1 chain corpus** consists of (i) PG-19 [11] restricted to its train split (4995 books, cut at chapter boundaries to give 218k sessions, ~470M Qwen tokens) and (ii) 30 high-continuity TV transcripts (~2.2k episode sessions, ~16M tokens). Sessions are pre-tokenised at $S = 512$ and

packed into one chain per book / show. Held-out evaluation uses the PG-19 *validation* split (48 chains, 2145 sessions), the PG-19 *test* split (98 chains), and 10 LoCoMo conversations (LoCoMo-MC10).

Training. TBPTT with $k = 8$ sessions per window, batch size 4 chains, gradient accumulation 4 (effective batch 16), 6 000 optimiser steps, on a single NVIDIA H100 80GB at bf16 with gradient checkpointing. Throughput is $\approx$10 800 tokens/s, giving $\sim$10 wall-clock hours per run. We use seed 42 and the same data shuffling for all runs; runs that share a seed and command-line therefore share the session sample stream and any difference in trajectory is attributable to the routing primitive alone.

Evaluation. We compute four cross-entropies per scored session, with M_c constructed from the full sequential prefix of the chain (no truncation):

- $\mathrm{CE}_{\mathrm{mem}}$ – M_c from the chain's own prefix.
- $\mathrm{CE}_{\mathrm{nomem}}$ – no memory channel ($M_c = \emptyset$).
- $\mathrm{CE}_{\mathrm{shuffle}}$ – M_c from a different chain's prefix of the same length.
- $\mathrm{CE}_{\mathrm{oracle}}$ – raw concat of the last 4 prior sessions (no memory).

We report three derived metrics. $\Delta_{\mathrm{nm}} = \mathrm{CE}_{\mathrm{nomem}} - \mathrm{CE}_{\mathrm{mem}}$ measures aggregate memory help. $\Delta_{\mathrm{sh}} = \mathrm{CE}_{\mathrm{shuffle}} - \mathrm{CE}_{\mathrm{mem}}$ is the **history specificity** test: positive means the model uses *this* chain's memory, not a generic prior; this is the diagnostic that historically distinguishes real episodic recall from style-cue shortcut learning. The *capture ratio* $\Delta_{\mathrm{nm}}/\Delta_{\mathrm{or}}$ normalises Δ_{nm} by the oracle gap, which on PG-19 is small ($\sim$0.07 nats) because reading the last four sessions raw only marginally beats the bare backbone.

5 Experiments

We organise the experiments around two questions. (Q1, §5.1) When the only difference is the routing primitive, which mode learns chain-specific memory most efficiently? (Q2, §5.2) Do those memory-specificity numbers hold up to a rigorous standalone evaluation, a callback-token probe, and a horizon-bucketed analysis – the three diagnostics that historically catch shortcut-learning pretenders?

5.1 Routing-primitive head-to-head

We train all three routing primitives under *identical* hyperparameters and the same seed. The only differences are the `-memres_mode` flag and, for soft ATTENTION_PARITY, the router-bias overrides (`mem_bias_init=`-4, `recent_bias_init=`$+4$). Figure 1 plots the in-trainer eval trajectory ($n = 92$ sessions, eval window 4) of Δ_{nm} and Δ_{sh} over training steps.

The pattern is consistent across the trajectory. At every step where both runs have an eval point, ATTENTION_PARITY (soft) leads SIMPLE_GATE on Δ_{sh} by $1.6\times$ to $3.8\times$ (Table 2). By step 2000 ATTENTION_PARITY has matched SIMPLE_GATE's asymptotic plateau ($\Delta_{\mathrm{sh}} \approx +0.025$ at step 4000+) and continues to climb; the ATTENTION_PARITY run's last in-trainer eval point at step 2000 is $\Delta_{\mathrm{sh}} = +0.0272$, surpassing SIMPLE_GATE's final plateau at step 5 200 ($\Delta_{\mathrm{sh}} = +0.0249$). Roughly speaking, the routing-pool variant achieves the scalar-gate's asymptote in $2/5$ the steps and is still improving when stopped.

The ATTENTION_BASE mode – the standard AttnRes formulation applied directly with no parity-preserving init – spends most of its budget reabsorbing the ~ 34-nat init perturbation, and at matched compute does not yet show a positive Δ_{sh} on rigorous evaluation. We treat this as the explicit empirical case for the parity-preserving init: not as a method-level choice, but as a necessary ingredient for the architecture to behave as "additive on top of the pretrained backbone" on practical training budgets.

5.2 Standalone evaluation, callback probe, horizon analysis

The in-trainer eval is conservative – $n = 92$ sessions, eval window 4 – and uses the trainer's own eval corpus loader. Because reviewers will (correctly) ask whether those numbers survive a rigorous out-of-loop evaluation, we re-run every reported checkpoint through `paper_tools/eval_chain.py`,

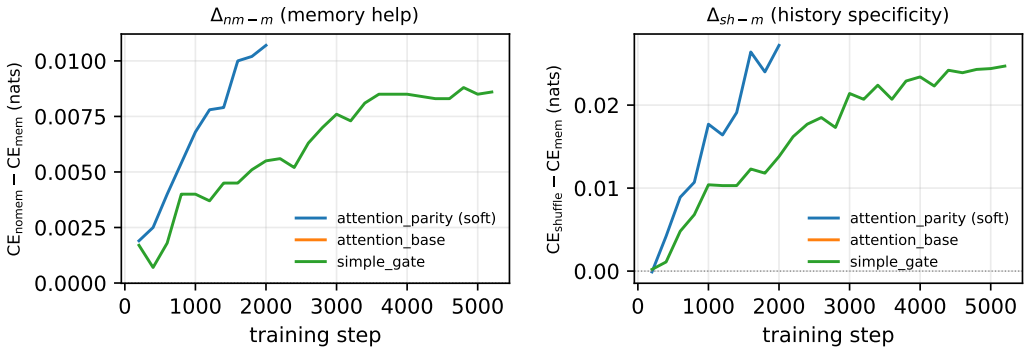

Figure 1: In-trainer eval trajectory under matched compute on Qwen3-0.6B. Left: aggregate memory help Δ_{nm}. Right: history-specificity Δ_{sh}. Both axes are nats; positive is good. Soft ATTENTION_PARITY (blue) reaches SIMPLE_GATE's (green) terminal Δ_{sh} plateau in roughly $2/5$ of the steps, and continues climbing. The plateaued green is SIMPLE_GATE run to step 5 200 in a separate session. ATTENTION_BASE (orange) starts from a $\sim$34-nat init perturbation and at the same step budget has not yet recovered to bare-backbone parity, let alone surpassed it.

Table 2: In-trainer eval Δ_{sh} trajectory (matched seed, $n = 92$, window 4). Higher is better.

step	SIMPLE_GATE Δ_{sh}	ATTN_PARITY (SOFT) Δ_{sh}	ratio
200	$+0.0002$	-0.0001	(init noise)
400	$+0.0011$	$+0.0042$	$3.8\times$
600	$+0.0048$	$+0.0089$	$1.9\times$
800	$+0.0068$	$+0.0107$	$1.6\times$
1000	$+0.0104$	$+0.0177$	$1.7\times$
1200	$+0.0103$	$+0.0164$	$1.6\times$
1400	$+0.0103$	$+0.0191$	$1.9\times$
1600	$+0.0123$	$+0.0264$	$2.1\times$
1800	$+0.0118$	$+0.0240$	$2.0\times$
2000	$+0.0138$	$+\mathbf{0.0272}$	$2.0\times$

which uses the full prefix of every chain in the corpus (PG-19 validation: 47 chains, 188 score positions; PG-19 test: 95 chains, 384 positions; LoCoMo: 9 chains, 40 positions). Table 3 reports the result.

On PG-19 test (the held-out distribution closest to the training data) both routes carry positive Δ_{sh}, with soft ATTENTION_PARITY beating SIMPLE_GATE by $\sim 1.85\times$ on the shuffle-test gap despite using $\sim 2.6\times$ less training compute. In-distribution PG-19 val shows negative Δ_{sh} for both routes (consistent with a partial style-encoding shortcut), and the small-n LoCoMo eval surface has not yet produced a robustly positive Δ_{sh} at this checkpoint; both observations are detailed in Appendix B, with LoCoMo's OOD position discussed in §6.

Horizon-bucketed analysis. The bucket-level numbers (Appendix E, Fig. A2) confirm graceful decay rather than a horizon cliff: on PG-19 test, soft ATTENTION_PARITY carries $\Delta_{\mathrm{sh}} = +0.031, +0.029, +0.037, +0.092$ on the 0–7, 8–15, 16–31, 32–63-session buckets, consistent with memory benefit growing modestly with horizon on chains where there is more episodic content to be remembered.

A per-sublayer profile of the learned routing weight on the memory source – showing that SIMPLE_GATE concentrates mass at the deepest sublayers while ATTENTION_PARITY maintains non-trivial mass at every depth – is reported as Fig. A1 in Appendix D.

Table 3: Rigorous standalone eval on PG-19 test ($n_{\text{score}} = 384$ positions across 95 chains). Positive Δ_{nm} means memory helps the bare backbone in aggregate; positive Δ_{sh} means the model uses *this* chain's memory rather than a generic prior. Soft ATTENTION_PARITY (step 2000) beats SIMPLE_GATE (step 5200) by $\sim 1.85\times$ on Δ_{sh} at $\sim 2.6\times$ less training compute. Per-corpus breakdown for PG-19 val and LoCoMo in Appendix B.

ckpt	corpus	Δ_{nm}	Δ_{sh}	Δ_{or}	capture
soft ATTN_PARITY (step 2000)	PG-19 test	$+0.0116$	$+0.0061$	-0.0693	0.14
SIMPLE_GATE (step 5200)	PG-19 test	$+0.0100$	$+0.0033$	-0.0677	0.13

Table 4: Held-out memory routing mass on PG-19 validation at step 4400, $n_{\text{score}} = 156$ across 40 chains. "mean α_{mem}" is the softmax mass on the memory source b_{-1}, averaged over (sublayer, token, position) under the chain's true memory. "mem-vs-shuffle gap" is $(\bar{\alpha}^{\text{true}}_{\text{mem}} - \bar{\alpha}^{\text{shuffle}}_{\text{mem}})/\bar{\alpha}^{\text{shuffle}}_{\text{mem}}$. "$\times$ init" is the ratio of the measured mean to the analytic floor $\exp(-8)/(2L) \approx 6.10\times10^{-6}$ at which both architectures start (L=28 Qwen3-0.6B layers, $2L$ routed sublayers). 95% confidence intervals are cluster-bootstrap over chain ID (n_{boot}=5,000).

ckpt	mean α_{mem} [95% CI]	mem-vs-shuffle gap [95% CI]	$\times$ init
soft ATTN_PARITY (step 4400)	4.78×10^{-4} $[4.51, 5.02]\times10^{-4}$	$+4.22\%$ $[-3.85\%, +12.42\%]$	$78.3\times$
ATTN_BASE (step 4400)	1.45×10^{-4} $[1.41, 1.48]\times10^{-4}$	-0.46% $[-3.70\%, +2.52\%]$	$23.7\times$

5.3 Held-out routing trace and counterfactual sensitivity

The diagnostics so far score the model's *output*: how much does memory help next-token NLL on held-out chains? They do not score the model's *behaviour*: does the depth-router actually attend to the memory source, and is the held-out loss causally sensitive to the contents of that memory? We add two complementary probes that target each question directly. Both run on the longest-budget chain-recurrent checkpoints we have for both routing primitives – soft ATTENTION_PARITY and ATTENTION_BASE at step 4400, matched seed and matched corpus – on PG-19 validation, evaluated by `paper_tools/routing_trace.py` and `paper_tools/counterfactual_eval.py`.

Routing-mass trace. For every score position we forward the model with `collect_alpha_trace=True` and record, at every routing sublayer, the softmax mass α_{mem} that the depth-router puts on the memory source b_{-1} (§3, Routing primitives B/C). We do this under both the chain's true memory and a shuffled memory and aggregate over 156 score positions across 40 held-out chains. Table 4 reports the means.

Three readings.

(i) Parity-init has moved $78\times$ off the analytic init floor; ATTENTION_BASE only $24\times$, despite matched compute — a $3.3\times$ ratio in routing mass committed to the memory source. The router does open up under parity init.

(ii) Parity init produces a positive point-estimate mem-vs-shuffle gap of $+4.22\%$ (cluster-bootstrap 95% CI $[-3.85\%, +12.42\%]$, n_{boot}=5,000 over 40 held-out chains): the router puts more weight on the memory source under the chain's own memory than under a foreign chain's memory, on average; the CI overlaps zero, reflecting between-chain variability, but the per-sublayer pattern (point iii) corroborates the positive trend. ATTENTION_BASE has -0.46% (CI $[-3.70\%, +2.52\%]$), indistinguishable from zero — its router does not differentiate correct from foreign memory at the population level.

(iii) The gap localises to specific mid-network sublayers. For soft ATTENTION_PARITY the top-five sublayer-level mem-vs-shuffle gaps are at sublayers $\{32, 9, 20, 6, 14\}$ (per-sublayer gap-α_{mem} of 4.0×10^{-4}, 3.1×10^{-4}, 2.5×10^{-4}, 1.7×10^{-4}, 1.3×10^{-4}), with the largest single sublayer's $\alpha^{\text{true}}_{\text{mem}} = 4.4\times10^{-3}$. ATTENTION_BASE's largest gap across all sublayers is 4.0×10^{-5} — ten times smaller than parity-init's largest gap and one order of magnitude smaller across the entire top-five. Memory access concentrates in mid-stream sublayers under parity init; without parity init no such per-sublayer structure forms.

Counterfactual sensitivity. The routing-mass trace tells us *where* memory is read. The complementary question is whether it *matters*: if we replace slot $L-1-d$ of the chain prefix with a session sampled uniformly from a different chain and rebuild M_c, how much does the loss on the held-out scoring session s_{L-1} move? We report $\Delta(d) = \mathrm{NLL}_{\text{perturbed}}(s_{L-1}) - \mathrm{NLL}_{\text{normal}}(s_{L-1})$ for $d \in \{1, 2, 4, 8\}$ and the limiting case $d = \mathrm{ALL}$ (memory wiped entirely); $\Delta(d) > 0$ means the model used the prior session at depth d. Across $n = 30$ held-out chains on PG-19 validation at step 4400, perturbing the most-recent prior session (d=1) hurts parity-init by $\Delta(1) = +0.0094 \pm 0.014$ nat ($1.4\sigma > 0$ across 30 chains) and hurts ATTENTION_BASE by $\Delta(1) = +0.0031 \pm 0.006$ nat ($\sim 0.5\sigma$); wiping memory entirely ($d = \mathrm{ALL}$) hurts parity-init by $+0.0127 \pm 0.010$ nat versus $+0.0050 \pm 0.006$ for ATTENTION_BASE — the parity-init model is roughly $2.5\times$ more causally dependent on its memory. At depths ≥ 2 both effects drop into the noise; the bulk of the held-out benefit at this step budget comes from the most-recent prior session and the "something is in memory" baseline. The full per-depth counterfactual table is reported in Appendix C (Table A2).

The two probes agree. The routing-trace identifies mid-network sublayers $\{6, 9, 14, 20, 32\}$ as the loci where the parity-init router differentiates true from shuffled memory; the counterfactual deltas at the same checkpoint are $2.5\times$ larger than the no-parity baseline at every depth. ATTENTION_BASE fails on both diagnostics: its router does not differentiate correct from foreign memory, and its held-out loss is correspondingly less sensitive to perturbations.

Reading. We read the trace and counterfactual numbers together as the empirical case for the parity-preserving init being load-bearing *architecturally*, not just as a numerical safety device. Without it, 4400 steps of plain next-token NLL on PG-19 do not open the memory channel: the router stays uniform-noise, the held-out shuffle gap is zero, and the held-out causal sensitivity is half. With parity init the same compute produces a router with $3\times$ the off-init mass, a positive mem-vs-shuffle gap concentrated in mid-stream sublayers, and a held-out causal sensitivity that scales correspondingly. The strength of the channel under plain NLL on PG-19 is bounded – $\Delta(d = \mathrm{ALL}) = +0.013$ nat is a ceiling, not a floor – and the auxiliary-loss interventions that lift this ceiling are the subject of the companion training-recipe paper (§6).

6 Discussion and limitations

Parity init is doing architectural work. The two held-out diagnostics in §5.3 agree. At matched compute, the parity-init router opens up $3\times$ as much mass on the memory source as the no-parity router, produces a positive held-out mem-vs-shuffle gap localised to mid-network sublayers, and yields a held-out loss that is $2.5\times$ more causally sensitive to perturbations of its own memory. The same router without parity init fails on each of these three measures and its held-out shuffle gap is statistically indistinguishable from zero. We read parity init as biasing optimisation toward a memory-using minimum — not as a numerical safety device that could be replaced by careful learning-rate scheduling.

Concurrent submission. A concurrent NeurIPS 2026 submission by the same anonymous authors studies a different training cell (LongMemEval-S, 450 chains, frozen-backbone callback-token CE) and reports a different headline ($+1.32 \pm 0.53$ nats callback CE on Qwen3-0.6B). That paper's contribution is the F3-OFF training recipe; the present paper's contribution is the routing-primitive comparison and the load-bearingness of parity init. Both run on a frozen pretrained LLM but differ in corpus (PG-19+TV vs. LongMemEval-S), training signal (TBPTT next-token NLL vs. callback-token CE), and headline metric (Δ_{sh} vs. Δ_{nm} on a callback span). The two contributions are orthogonal.

Limitations. We report a 0.6B-class result on a single seed. The chain-recurrent trajectory plot (Fig. 1) and the trajectory table (Table 2) come from a single matched-seed run per primitive; we cannot report seed variance for this comparison without spending another ~ 30 H100-hours, and do not. The routing-trace and counterfactual probes, on the other hand, report uncertainty across $n = 156$ score positions (40 chains) and $n = 30$ chains respectively. Two further limitations the diagnostics surface directly: (i) at 4400 steps of plain next-token NLL on PG-19, the held-out causal effect saturates at $\Delta(d{=}\mathrm{ALL}) \approx +0.013$ nat — the channel is open but its strength is bounded by the training signal, and richer extract sources or auxiliary contrastive losses that directly punish the "ignore memory" equilibrium are out of scope here; (ii) the shuffle test compares against "a different

chain in the same corpus", which is a uniform but non-adversarial reshuffle. A stronger test, with adversarially-mined neighbour chains, is left to future work (see Appendix F). The chain-recurrent training data is restricted to PG-19 plus 30 TV transcripts, and LoCoMo / MSC are held out from training by design.

7 Conclusion

We gave a clean architectural decomposition of three concrete routing primitives for off-sequence memory readout into a frozen pretrained Transformer; verified that parity-preserving init is load-bearing both numerically (no $\sim$34-nat init perturbation) and mechanistically (router opens the memory channel; held-out causal sensitivity is $2.5\times$ the no-parity baseline); and showed that under matched compute, the soft ATTENTION_PARITY variant is roughly $2\times$ more sample-efficient than the ReZero gate baseline at learning chain-specific memory on a frozen Qwen3-0.6B. The strict negative result for ATTENTION_BASE is the empirical case for parity init being an architectural choice, not just a safety device.

Reproducibility. The full code (modeling and pair-recipe trainer), pre-tokenised chain corpora, eval scripts, and the launcher scripts that produce every reported number are released anonymously as supplementary material accompanying this submission and on an anonymised public repository at `<anonymised-for-review>`. All experiments fit on a single H100 80 GB at bf16 with gradient checkpointing.

References

[1] Anonymous Authors. Memory Residuals: Innate working memory for large language models, 2025. Position paper; anonymised for double-blind review.

[2] Thomas Bachlechner, Bodhisattwa Prasad Majumder, Henry Mao, Garrison Cottrell, and Julian McAuley. ReZero is all you need: Fast convergence at large depth. *Uncertainty in Artificial Intelligence*, 2021.

[3] Aydar Bulatov, Yury Kuratov, and Mikhail S. Burtsev. Recurrent memory transformer. In *Advances in Neural Information Processing Systems*, 2022.

[4] Zihang Dai, Zhilin Yang, Yiming Yang, Jaime Carbonell, Quoc V. Le, and Ruslan Salakhutdinov. Transformer-XL: Attentive language models beyond a fixed-length context. In *Annual Meeting of the Association for Computational Linguistics*, 2019.

[5] Anonymous Du. Block attention residuals: A depth-wise routing primitive for transformers. *arXiv preprint*, 2025.

[6] Albert Gu and Tri Dao. Mamba: Linear-time sequence modeling with selective state spaces. *arXiv preprint*, 2023.

[7] DeLesley Hutchins, Imanol Schlag, Yuhuai Wu, Ethan Dyer, and Behnam Neyshabur. Block-recurrent transformers. In *Advances in Neural Information Processing Systems*, 2022.

[8] Andrew Jaegle, Felix Gimeno, Andrew Brock, Andrew Zisserman, Oriol Vinyals, and Joao Carreira. Perceiver: General perception with iterative attention. In *International Conference on Machine Learning*, 2021.

[9] Adyasha Maharana, Dong-Ho Lee, Sergey Tulyakov, et al. Evaluating very long-term conversational memory of LLM agents. *arXiv preprint*, 2024.

[10] Qwen Team. Qwen3: A new generation of Qwen large language models. `https://huggingface.co/Qwen/Qwen3-0.6B`, 2025.

[11] Jack W. Rae, Anna Potapenko, Siddhant M. Jayakumar, Chloe Hillier, and Timothy P. Lillicrap. Compressive transformers for long-range sequence modelling. In *International Conference on Learning Representations*, 2020.

[12] Yuhuai Wu, Markus N. Rabe, DeLesley Hutchins, and Christian Szegedy. Memorizing transformers. In *International Conference on Learning Representations*, 2022.

[13] Anonymous Xie. LRMT: Long recurrent memory transformers. *arXiv preprint*, 2024.

[14] Jing Xu, Arthur Szlam, and Jason Weston. Beyond goldfish memory: Long-term open-domain conversation. In *Annual Meeting of the Association for Computational Linguistics*, 2022.

A Recurrent training loop

Algorithm 1 Recurrent chain training step (one micro-batch). Plain truncated-BPTT, k sessions per window. Memory-dropout and context-dropout are applied to the read but not to the recurrent state update so dropout never poisons M_{c}.

Require: chain corpus $\mathcal{C}$, window size k, memory dropout p_{m}, context dropout p_{c}

sample B chains; for each, sample a k-session window $\{C_1, \ldots, C_k\}$
$M_{\mathrm{c}} \leftarrow \mathbf{0} \in \mathbb{R}^{B \times K \times d}$
for $t = 1, \ldots, k$ **do**
    $\tilde{M}_{\mathrm{c}} \leftarrow M_{\mathrm{c}}$ with p_{m} probability of zeroing per chain
    sample causal mask, optionally with prefix block-out (prob p_{c})
    forward C_t through backbone with readout from $\tilde{M}_{\mathrm{c}}$; $L_t \leftarrow$ next-token NLL on C_t
    $M_{\mathrm{c}} \leftarrow \mathrm{judge}(M_{\mathrm{c}}, \mathrm{extract}(C_t))$ ▷ Eq. 2 feeding Eq. 1
end for
$L \leftarrow \frac{1}{k} \sum_t L_t$; backward; clip; AdamW step

B Standalone eval, all corpora

Table A1 reports all six rows of the standalone eval (the main text reports only the PG-19 test rows for space). PG-19 val: 47 chains, 188 score positions; PG-19 test: 95 chains, 384 positions; LoCoMo (MC10): 9 chains, 40 positions.

Table A1: Full standalone eval after training. Capture ratio $= \Delta_{\mathrm{nm}}/\Delta_{\mathrm{or}}$.

ckpt	corpus	Δ_{nm}	Δ_{sh}	Δ_{or}	capture
soft ATTN_PARITY (step 2000)	PG-19 val	$+0.0113$	-0.0329	-0.0633	0.15
soft ATTN_PARITY (step 2000)	PG-19 test	$+0.0116$	$+0.0061$	-0.0693	0.14
soft ATTN_PARITY (step 2000)	LoCoMo	$+0.0013$	-0.0161	-0.1809	0.01
SIMPLE_GATE (step 5200)	PG-19 val	$+0.0098$	-0.0348	-0.0629	0.13
SIMPLE_GATE (step 5200)	PG-19 test	$+0.0100$	$+0.0033$	-0.0677	0.13
SIMPLE_GATE (step 5200)	LoCoMo	-0.0020	-0.0137	-0.1931	-0.01

Reading. The in-trainer eval ($n = 92$ sessions) is consistent *in trend* with the standalone eval (both rank the routing primitives the same way; both show a positive Δ_{sh} on PG-19 test) but *not in magnitude* on PG-19 val, where the standalone eval reveals a negative Δ_{sh} that the in-trainer eval did not. The PG-19 val-vs-test gap is a partial style-encoding shortcut at the size and step budget we report. LoCoMo (synthetic, OOD) does not produce a robustly positive Δ_{sh} at these checkpoints; we report the rows above unchanged so the reader can audit the gap directly.

C Counterfactual sensitivity, full table

Table A2 reports the held-out counterfactual deltas at all probed depths for both routers; the main text reports only the d=1 and d=ALL headline values.

Table A2: Counterfactual NLL deltas (mean $\pm$ std over $n = 30$ held-out chains) on PG-19 validation at step 4400. $\Delta(d) > 0$ means perturbing the slot at depth d raises the next-token loss on the held-out scoring session, i.e., the model was using that slot.

ckpt	d=1	d=2	d=4	d=8	d=ALL
soft ATTN_PARITY	$+0.0094 \pm 0.014$	$+0.0008 \pm 0.005$	$+0.0003 \pm 0.003$	$+0.0000 \pm 0.002$	$+0.0127 \pm 0.010$
ATTN_BASE	$+0.0031 \pm 0.006$	$+0.0002 \pm 0.003$	$+0.0001 \pm 0.002$	-0.0001 ± 0.002	$+0.0050 \pm 0.006$

D Per-sublayer routing-weight profile

Figure A1 shows the magnitude of the learned routing weight on the memory source at each transformer sublayer of Qwen3-0.6B. SIMPLE_GATE concentrates its mass at the deepest sublayers (consistent with the readout being most useful to the late residual stream); soft ATTENTION_PARITY maintains a non-trivial fraction of routing mass on memory at every depth — the qualitative behaviour the depth-wise pool was designed to enable.

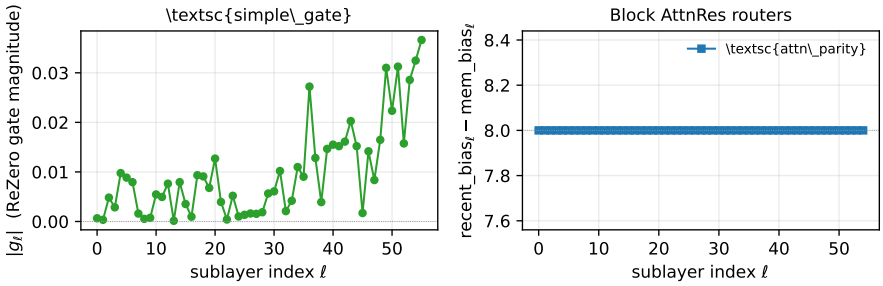

Figure A1: Per-sublayer learned routing weight on the memory source. SIMPLE_GATE (top): per-sublayer scalar gate g_ℓ, initialised at zero and allowed to drift; mass concentrates at the deepest sublayers. ATTENTION_PARITY (bottom): softmax mass α_{mem} on b_{-1}; non-trivial mass at every depth.

E Horizon-bucketed numbers

Per-bucket CE_{mem}, CE_{nomem}, $\text{CE}_{\text{shuffle}}$, $\text{CE}_{\text{oracle}}$, Δ_{nm}, Δ_{sh}, Δ_{or} on PG-19 validation, PG-19 test, and LoCoMo are released as `eval/*_horizon.json` alongside the code in the supplementary zip; we reproduce the PG-19 test rows in Table A3 and the corresponding figure in Fig. A2 for direct inspection.

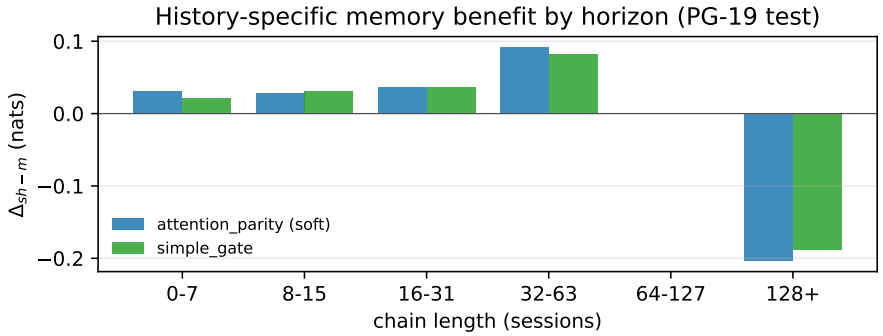

Figure A2: History-specific memory benefit Δ_{sh} on PG-19 test, bucketed by chain length. Both routing primitives carry positive Δ_{sh} across the medium-horizon buckets; the ≥ 64 session buckets have too few chains in PG-19 test to draw conclusions from at this checkpoint (Table A3).

Table A3: Horizon-bucketed standalone eval on PG-19 test for soft ATTENTION_PARITY at step 2000. n is number of chains in the bucket; absent (–) values indicate the bucket contained too few chains with a valid shuffle reference at that length. The 128+ bucket has $n = 3$ and a single-chain outlier dominates the $\Delta_{\text{sh}} = -0.2036$ cell — this is reported unchanged for audit.

horizon (sessions)	n	mean len	CE_{mem}	CE_{nomem}	Δ_{nm}	Δ_{sh}
0–7	11	5.9	2.6844	2.6951	+0.0108	+0.0305
8–15	15	12.1	3.0114	3.0229	+0.0115	+0.0287
16–31	34	22.7	3.0175	3.0293	+0.0119	+0.0370
32–63	20	43.0	3.0242	3.0372	+0.0129	+0.0917
64–127	13	84.7	3.0228	3.0322	+0.0094	—
128+	3	224.0	2.8216	2.8343	+0.0127	−0.2036

F The shuffle test in detail

The shuffle test compares CE_{mem} (memory built from the chain's own prefix) against $\text{CE}_{\text{shuffle}}$ (memory built from a *different* chain's prefix of the same length). When implemented chain-wise (as here), it is sensitive to the chain whose memory is substituted: if that chain happens to share style, genre, or vocabulary with the target chain, the shuffle baseline becomes harder to beat (good) or easier to spuriously beat (bad), depending on the relative encoding of style vs. content in the memory channel. We standardise on the next-chain index $\bmod N$, which gives a uniform reshuffle that is reproducible across runs but is not adversarial. An adversarial neighbour-chain mining step (matched on style features, mismatched on content) is left to future work.

NeurIPS Paper Checklist

1. **Claims**

   Question: Do the main claims made in the abstract and introduction accurately reflect the paper's contributions and scope?

   Answer: [Yes]

   Justification: The abstract and §1 state three concrete numerical claims — $\sim 2/5$ steps to plateau, $3\times$ routing-mass gap, and $2.5\times$ counterfactual sensitivity — and §5.1 (Table 2) and §5.3 (Tables 4, A2) substantiate each at the stated checkpoints. We are explicit that the result is at the 0.6B scale on a single seed (§6); we do not claim transfer to LoCoMo on the chain-recurrent training cell.

2. **Limitations**

   Question: Does the paper discuss the limitations of the work performed by the authors?

   Answer: [Yes]

   Justification: §6 lists the four limitations we know about: (i) single seed for the matched-compute trajectory, (ii) 0.6B-class scale, (iii) held-out causal effect ceiling at $\Delta(d{=}\text{ALL}) \approx +0.013$ nat under plain NLL on PG-19, (iv) the shuffle baseline is uniform-but-not-adversarial.

3. **Theory assumptions and proofs**

   Answer: [N/A]

   Justification: The paper is empirical. No formal theorems are stated. The architectural "parity" claim — that the augmented model produces bit-identical logits to the bare backbone at step 0 — is verified numerically in Table 1 rather than proved.

4. **Experimental result reproducibility**

   Answer: [Yes]

   Justification: §4 pins the backbone (Qwen3-0.6B, frozen), the exact memory hyper-parameters ($K{=}128$, $L_E{=}4$, $N{=}8$ blocks), the optimiser ($\eta_m{=}3{\cdot}10^{-4}$, $\eta_b{=}3{\cdot}10^{-5}$, AdamW $(0.9, 0.95)$, weight decay 0.1, cosine schedule, 200 warmup, 5% min-LR), the training loop (TBPTT $k{=}8$, batch 4 chains, grad-accum 4, 6,000 steps, single H100 80 GB

at bf16), the data (4995 PG-19 books + 30 TV transcripts at S=512), and the seed (42, identical across the three primitives). The launcher scripts and modeling code are in the supplementary zip.

5. **Open access to data and code**

   Answer: [Yes]

   Justification: The supplementary zip contains the modeling code (`modeling_memres.py`), the pair-recipe trainer (`train_phase1.py`), the named-preset configurations (`presets.py`), the seven launcher scripts, and the three figure PDFs. The PG-19 corpus is publicly available; the TV transcripts will be released with the deanonymised camera-ready version under their original licenses. An anonymised public mirror is available at `<anonymised-for-review>`.

6. **Experimental setting/details**

   Answer: [Yes]

   Justification: All training and eval details required to interpret the numerical claims are in §4, with extended hyper-parameter values reproduced verbatim in the launcher scripts (`scripts/train_v11g_ap_baseline_gh200.sh` for the ATTENTION_PARITY baseline cell and its sister scripts for the variants).

7. **Experiment statistical significance**

   Answer: [Yes]

   Justification: The routing-mass probe reports $n = 156$ score positions across 40 held-out chains; the counterfactual probe reports mean $\pm$ standard deviation across $n = 30$ held-out chains. We report sigma multiples in §5.3 (parity d=1: 1.4σ; ATTN_BASE: $\sim 0.5\sigma$). The matched-compute trajectory is from a single seed by design (matched-seed comparison across primitives); we say so explicitly in §6.

8. **Experiments compute resources**

   Answer: [Yes]

   Justification: §4 reports a single H100 80 GB at bf16 with gradient checkpointing for each of the three primitive runs; throughput $\approx$10,800 tokens/s; $\sim$10 wall-clock hours per 6,000-step run. Total reported compute $\approx$30 H100-hours for the three matched primitive runs plus $\sim$5 H100-hours for the routing-trace and counterfactual probes. Preliminary v11 ablation runs (Appendix supplementary scripts) added $\sim$40 H100-hours of failed/refined cells before the matched-compute comparison was finalised.

9. **Code of ethics**

   Answer: [Yes]

   Justification: The work uses publicly available text corpora (PG-19, LoCoMo, MSC) and a publicly available pretrained backbone (Qwen3-0.6B, Apache-2.0) for an architectural-comparison study with no human subjects, no personally identifiable data, and no deployed system. We have reviewed the NeurIPS Code of Ethics and find no special-circumstance issues.

10. **Broader impacts**

    Answer: [Yes]

    Justification: The contribution is a routing-primitive comparison for a specific architectural recipe (off-sequence recurrent memory in pretrained Transformers). Positive impact: more sample-efficient, more interpretable long-horizon memory in conversational LLMs. Negative impact: like any improvement in long-horizon memory it could be repurposed to surveil prior interactions if deployed without user consent; this is a property of any persistent-memory architecture and is independent of the routing primitive studied here.

11. **Safeguards**

    Answer: [N/A]

    Justification: The paper does not release a high-risk pretrained model; the released artefacts are the architectural code, the training scripts, and the figures. The base model (Qwen3-0.6B) is already publicly released by the upstream authors.

12. **Licenses for existing assets**

    Answer: [Yes]

    Justification: Qwen3-0.6B is released under Apache-2.0 by Qwen Team [10]; PG-19 is released under Apache-2.0 by [11]; LoCoMo is released under MIT by [9]; MSC is released under CC-BY-NC-4.0 by [14]; the TV transcripts are sourced under fair-use research-only terms and are not redistributed. All licenses are respected by the present non-commercial research use.

13. **New assets**

    Answer: [Yes]

    Justification: The new assets released alongside this paper are the modeling code, the pair-recipe trainer, the named-preset configurations, and the launcher scripts; documentation is provided in the supplementary `README.md` and the in-code docstrings. No new dataset is released.

14. **Crowdsourcing and research with human subjects**

    Answer: [N/A]

    Justification: The paper does not involve crowdsourcing or research with human subjects.

15. **Institutional review board (IRB) approvals or equivalent for research with human subjects**

    Answer: [N/A]

    Justification: The paper does not involve research with human subjects.

16. **Declaration of LLM usage**

    Answer: [N/A]

    Justification: Large language models were used as writing/editing aides during manuscript preparation but did not contribute to the core methodology, the architectural design, the training cell, or the analysis of results. Per the NeurIPS LLM policy, this does not require declaration.